Swag Labs E2E project

1. Project objective

“We are going to automate the complete user journey of the Swag Labs application using Selenium WebDriver and build a maintainable automation framework around it.”

Then explain the business flow:

Login → Products → Product Selection → Cart → Checkout → Order Completion → Logout

The important point is:

We are not just writing Selenium scripts. We are building an automation framework.

---

2. Explain the project in 4 layers

SWAG LABS AUTOMATION PROJECT

|

| | |

Test Layer Framework Layer Utility Layer

| | |

TestNG POM Excel

Tests Base Class Properties

Config Screenshots

Driver

Explain:

Test Layer

Contains **what we want to test**.

Example:

- Login test
- Product test
- Cart test
- Checkout test

Framework Layer

Contains **how the automation works**.

Example:

- Browser setup
- Driver management

- Page Object Model
- TestNG execution
- Configuration

Utility Layer

Contains reusable supporting functionality.

Example:

- Excel reading
- Properties reading
- Screenshot
- Wait utilities
- Reporting

---

3. Explain why we need a framework

Before showing the framework,

Suppose we have **100 test cases**.

If every test contains:

- Open browser
- Open URL
- Login
- Find elements
- Perform actions
- Close browser

then we will have a huge amount of duplicate code.

“What happens if Chrome Driver setup changes?”

They will understand that we would need to modify many tests.

Then introduce the framework:

“A framework provides a common structure where reusable functionality is created once and consumed by multiple test cases.”

This is one of the most important concepts of the session.

---

4. Explain the project architecture

For example:

SwagLabsAutomation

│

├── src/test/java

```
|   |
|   ├── base
|   |    └── BaseClass
|   |
|   ├── pages
|   |    ├── LoginPage
|   |    ├── ProductsPage
|   |    ├── CartPage
|   |    └── CheckoutPage
|   |
|   ├── tests
|   |    ├── LoginTest
|   |    ├── ProductTest
|   |    └── CheckoutTest
|   |
|   └── utilities
|        ├── ExcelUtils
|        ├── ConfigReader
|        └── ScreenshotUtils
|
├── src/test/resources
|   ├── config.properties
|   └── testdata.xlsx
|
└── testng.xml
```

Explain **why each folder exists**.

---

5. Explain Base Class

“The Base Class contains common setup and teardown functionality required by multiple test classes.”

Typical responsibilities:

BaseClass

↓

Browser initialization

↓

Open application

↓

Provide WebDriver

↓

Test execution

↓

Close browser

Why shouldn't we initialize ChromeDriver separately in every test class?

Answer:

Because browser setup is common functionality and should be centralized.

---

6. Explain Page Object Model

This should be a major part of the session.

“Page Object Model separates the application page structure from the test logic.”

For Swag Labs:

Login Page

↓

Products Page

↓

Cart Page

↓

Checkout Page

↓

Order Confirmation

Each page gets its own Page Class.

For example:

LoginPage

- username locator
- password locator
- login button
- login action

Then:

LoginTest

↓

LoginPage

↓

Swag Labs Application

The **test class says what to test**, while the **page class handles how to interact with the page**.

That distinction is very important for students.

---

7. Explain the complete E2E flow

Now show them the actual business flow:

LOGIN

↓

PRODUCTS PAGE

↓

SELECT PRODUCTS

↓

ADD TO CART

↓

CART PAGE

↓

CHECKOUT

↓

ENTER CUSTOMER DATA

↓

REVIEW ORDER

↓

COMPLETE ORDER

↓

ORDER CONFIRMATION

↓

LOGOUT

Explain that this is an **E2E test scenario** because we are validating the complete business journey rather than one isolated feature.

---

8. Explain TestNG

After understand the application flow, introduce TestNG.

Explain:

“Selenium performs browser automation. TestNG provides the test execution framework.”

Then explain the responsibilities:

Selenium

↓

Browser automation

TestNG

↓

Test execution

↓

Annotations

↓

Assertions

↓

Test priority

↓

DataProvider

↓

Reports

Important distinction:

Selenium ≠ Testing Framework

Selenium automates the browser.

TestNG manages and executes the tests.

---

9. Explain configuration management

Introduce the .properties file.

Example conceptually:

browser = chrome

url = https://www.saucedemo.com

username = standard_user

password = secret_sauce

Explain why we don't want these values hardcoded everywhere.

For example:

Test Code

↓

ConfigReader

↓

config.properties

Benefits:

- Easy maintenance
- Environment changes
- Less hardcoding
- Reusability

---

10. Explain Data-Driven Testing

Then introduce Excel.

Suppose we have:

Username	Password	Expected Result
standard_user	secret_sauce	Success
invalid_user	wrong123	Failure

Explain:

“Instead of creating separate test methods for every data combination, we separate test data from test logic.”

Architecture:

Excel

↓

Excel Utility

↓

DataProvider

↓

Test Method

↓

Application

This naturally leads into **DataProvider + Excel + TestNG**.

---

11. Explain Assertions

Should understand that automation isn't just clicking buttons.

For example:

Login

↓

Products page

↓

Verify Products page is displayed

Explain:

“An action tells the application to do something. An assertion verifies whether the application behaved as expected.”

Examples of validations:

- Verify login successful
- Verify Products page displayed
- Verify product added to ca0.rt
- Verify cart count
- Verify checkout page
- Verify order confirmation
- Verify logout

---

12. Explain reusable utilities

Show the framework concept:

```
            UTILITIES
               |
   ---------------------------------
   |           |           |
  Excel       Config      Screenshot
  Utility     Reader       Utility
```

Here:

Excel Utility

Reads test data.

Config Reader

Reads configuration values.

Screenshot Utility

Captures screenshots when required.

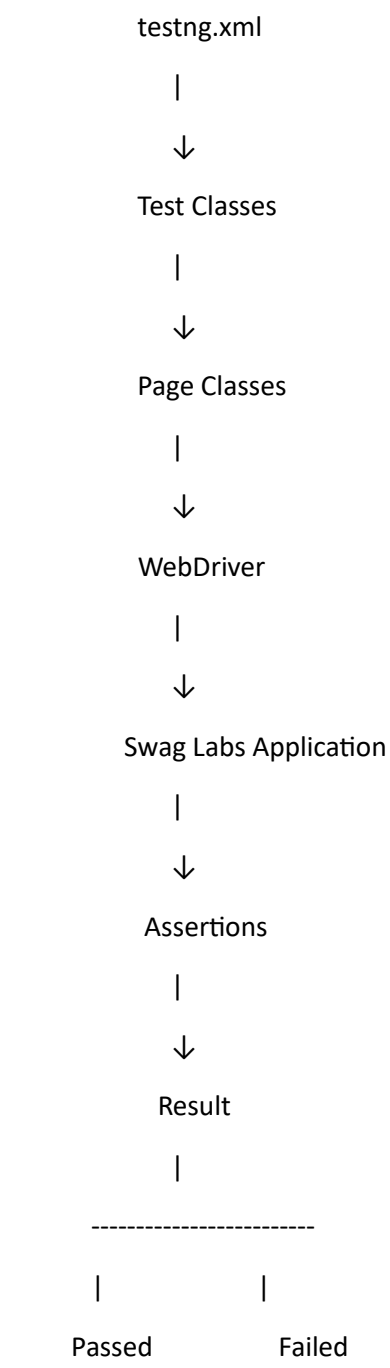
The key message:

“Utilities contain common functionality that can be reused across different test cases.”

---

13. Explain the complete execution architecture

This is a very effective diagram for students:



Then add:

Excel ————→ DataProvider ——→ Test

Properties —→ ConfigReader ——→ Framework

---

14. Explain the project from a tester's perspective

Real automation project normally goes through:

Requirement

↓

Test Scenario

↓

Test Case

↓

Automation Decision

↓

Framework Design

↓

Script Development

↓

Test Execution

↓

Validation

↓

Defect Reporting

↓

Regression Execution

↓

Test Report

This helps them understand **where Selenium fits into the overall QA process**.

---

15. Then explain advanced framework concepts

Once the basic architecture is understood, move to:

Level 1 – Selenium

WebDriver

Locators

WebElement

Actions

Screenshots

Level 2 – TestNG

Annotations

Assertions

DataProvider

testng.xml

Groups

Priority

Reports

Level 3 – POM

Page Classes

Locators

Page Methods

Reusable components

Level 4 – Data Driven

Excel

Properties

DataProvider

Level 5 – Framework

Base Class

Utilities

Configuration

Logging

Screenshots

Reports

Level 6 – CI/CD

Git

GitHub

GitHub Actions

Maven

Automated execution

---

16. Students should be able to answer

1. **Why do we need a framework?**
2. **Why do we use Base Class?**
3. **What is the purpose of Page Object Model?**
4. **What is the difference between Test Class and Page Class?**
5. **Why do we use TestNG?**

6. **Why do we separate test data from test logic?**
7. **Why do we use Excel?**
8. **Why do we use properties files?**
9. **Where should screenshots be implemented?**
10. **What is the complete Swag Labs E2E flow?**
11. **What is reusable code in our framework?**
12. **How would you execute the entire project?**
13. **How would you execute only failed tests?**
14. **How would you execute tests in parallel?**
15. **How would you run this framework through Jenkins/GitHub Actions?**

Best teaching sequence

I recommend explain the project in this exact order:

Application → Business Flow → Test Scenarios → Selenium → TestNG → Framework Need → Project Structure → Base Class → POM → Utilities → Properties → Excel/DataProvider → Assertions → Reports → Maven → Git → CI/CD

→ "Explain your Selenium framework."

A strong answer would be:

"I worked on a Selenium Java automation framework for an e-commerce application. We used Selenium WebDriver for browser automation, TestNG for test execution, Page Object Model for maintaining page-specific locators and actions, Maven for build and dependency management, Excel for data-driven testing, and properties files for configuration management. We had a Base Class for common setup and teardown and reusable utilities for Excel, screenshots and configuration. The test cases covered the complete Swag Labs E2E flow from login to checkout and order confirmation. The project was maintained in Git, and the Maven suite could be integrated with Jenkins or GitHub Actions for CI/CD and automated regression execution

****